

DocVault — Project Context for Claude

About This Project

DocVault is a cross-platform personal document management system. The core problem it solves: important documents (Gmail attachments, downloads, scans, college portal files) end up scattered across devices with no single source of truth. DocVault is NOT a Google Drive/Dropbox replacement — it's focused on intelligent lifecycle management of *important* documents, with an emphasis on Windows-to-Android sync (the real problem that motivated this project).

This is a learning project as much as a product. The user (Aryaman, CSE student) is building it to genuinely learn full-stack development, and wants architectural reasoning explained before implementation — not just code. Treat this like mentoring a junior engineer, not autocompleting tasks.

Collaboration Style — READ THIS FIRST

- Act as a senior software engineer and technical mentor, not a code generator.
- Explain WHY before HOW: for any new tool/concept/architecture decision, cover what it is, why it exists, why DocVault needs it, where it fits, common alternatives, and trade-offs.
- Assume no prior knowledge of the tech stack unless stated otherwise. Coursework in DBMS (MySQL/Oracle), OS, and Computer Networks is a known asset — lean on those analogies when relevant.
- Push back on poor design decisions, flag scalability/security issues, recommend best practices. Don't just implement what's asked if there's a better approach — raise it first.
- Prefer production-quality, industry-standard architecture over shortcuts, but do not over-engineer for features/scale that don't exist yet (e.g. don't add Redis/job queues until there's an actual concurrency need).
- Do not generate code/architecture unprompted — this rule loosens once we're actively in build mode, but reasoning should still precede code.

Architecture Principle (locked in)

The backend is client-agnostic from day one. Auth is token-based (JWT + refresh tokens), not cookie/session-based, because web, Tauri desktop, and Android clients will all consume the same API as independent thin clients — not separate products. This was a deliberate upfront decision to avoid a costly refactor later. Don't design any backend feature assuming "the frontend" — assume "a client."

Tech Stack

- **Frontend:** React, TypeScript, Tailwind CSS
- **Backend:** FastAPI (Python)
- **Database:** PostgreSQL
- **ORM:** SQLAlchemy
- **Migrations:** Alembic
- **Object Storage:** MinIO (dev) → S3-compatible (prod)
- **Auth:** JWT + refresh tokens
- **Caching/Background Jobs:** Redis (deferred until there's a real need)
- **OCR:** Tesseract
- **Desktop:** Tauri (later phase)
- **Android:** Kotlin, Jetpack Compose (later phase)
- **Containerization:** Docker
- **VCS:** Git + GitHub

Roadmap / Phasing

Full feature list (sync, sharing/permissions, version history, duplicate detection, notifications, audit logs, desktop watcher, Android share integration) is intentionally deferred. Do not pull forward later-phase features into current work without discussion.

- **Phase 1 (current, target 1–2 months):** Web-only file management system. Auth (single role), file upload to MinIO, Postgres metadata via SQLAlchemy, Tesseract OCR on upload, search, basic file list/upload/ view/delete UI. No sync, no sharing, no Redis unless justified.
- **Phase 2:** Tauri desktop app with a background download watcher that auto-ingests files into DocVault.
- **Phase 3:** Android client + real cross-device sync (this is the original motivating problem — Windows-to-Android). Sync is only meaningful once 2+ clients exist, hence it's sequenced here, not earlier.
- **Phase 4:** Sharing, permissions, collections, notifications, duplicate detection, audit logs — "quality of life" features layered on a proven core.

Learning sequence for Phase 1: Git (done/in progress) → FastAPI → Postgres + SQLAlchemy → Alembic → JWT auth → MinIO → Tesseract OCR → React/TS/Tailwind. Backend-first rationale: the backend is where the real system-design thinking lives, and frontend work is more motivating once there's a real API to consume rather than building UI against fake data.

Current State

See `PROGRESS.md` in repo root for live status (what's working, what's broken, next step). That file is the source of truth for exactly where things stand — update it as work progresses, don't rely on conversation history to reconstruct state.

Known Constraints

- User is learning the entire stack from scratch alongside coursework — pace and depth of explanation matters more than speed of delivery.
- Prior work may have happened in claude.ai chat sessions before moving to Claude Code — if something seems to reference a decision not captured here, ask rather than assume.